Operating Systems

Mahsa Ziraksima
ACIBADEM UNIVERSITY
Spring 2025

7. Scheduling: Introduction

Scheduling: Introduction

high-level policies that an OS scheduler employs:
scheduling policies or disciplines

- Workload or process running assumptions:

  (fully-operational scheduling discipline)

  1. Each job runs for the **same amount of time.**
  2. All jobs **arrive** at the same time.
  3. All jobs only use the **CPU** (i.e., they perform no I/O).
  4. The **run-time** of each job is known.

Scheduling Metrics

- Performance metric: Turnaround time
  - The time at which **the job completes** minus the time at which **the job arrived** in the system.

$$T_{turnaround} = T_{completion} - T_{arrival}$$

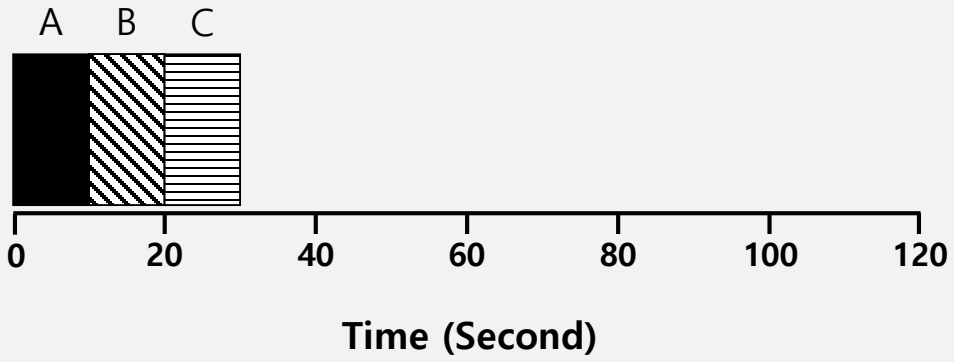
- All jobs arrive at the same time:

$$T_{arrival} = 0, \qquad T_{turnaround} = T_{completion}$$

- Another metric is fairness.
  - Performance and fairness are often at odds in scheduling.
  - a scheduler may optimize performance but at the cost of preventing a few jobs from running, thus decreasing fairness.

First In, First Out (FIFO)

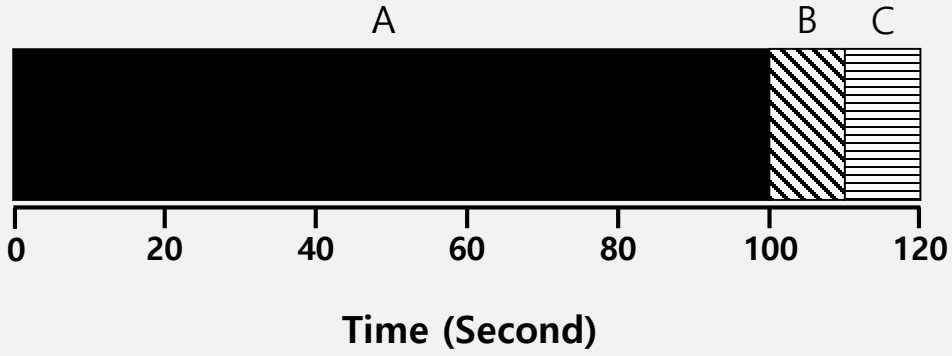
- First Come, First Served (FCFS)
  - Very simple and easy to implement
- Example:
  - A arrived just before B which arrived just before C.
  - Each job runs for 10 seconds.



$$Average\ turnaround\ time = \frac{10 + 20 + 30}{3} = 20\ sec$$

Why FIFO is not that great? – Convoy effect

- Let's relax assumption 1: Each job **no longer** runs for the same amount of time.
- Example:
  - A arrived just before B which arrived just before C.
  - A runs for 100 seconds, B and C run for 10 each.



$$Average\ turnaround\ time = \frac{100 + 110 + 120}{3} = 110\ sec$$

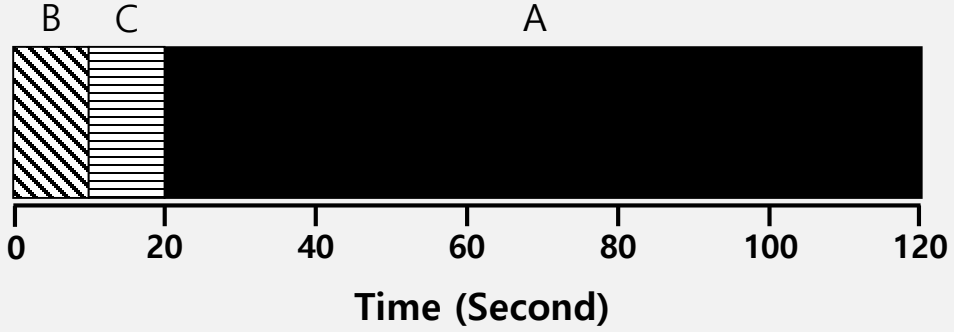
Why FIFO is not that great? – Convoy effect

- Convoy effect:

A number of relatively-short potential consumers of a resource get queued behind a heavyweight resource consumer.

Shortest Job First (SJF)

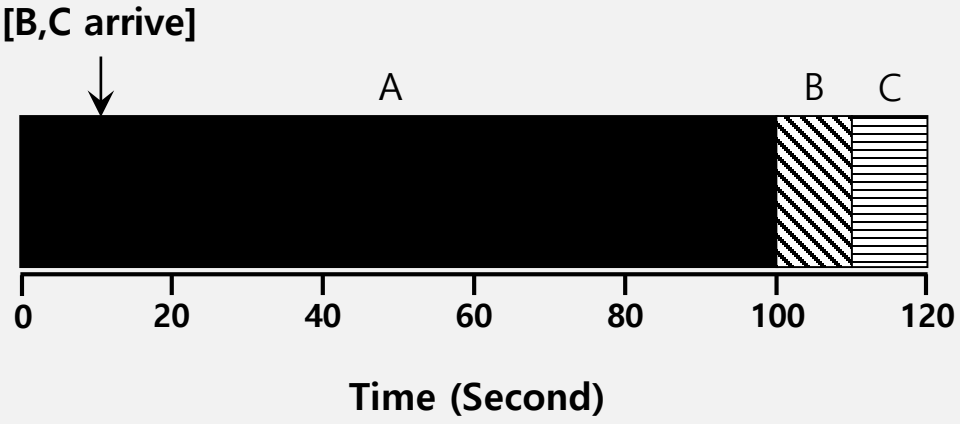
- Run the shortest job first, then the next shortest, and so on
  - Non-preemptive scheduler
- Example:
  - A arrived just before B which arrived just before C.
  - A runs for 100 seconds, B and C run for 10 each.



$$Average\ turnaround\ time = \frac{10 + 20 + 120}{3} = 50\ sec$$

SJF with Late Arrivals from B and C

- Let's relax assumption 2: Jobs can arrive at any time.
- Example:
  - A arrives at t=0 and needs to run for 100 seconds.
  - B and C arrive at t=10 and each need to run for 10 seconds



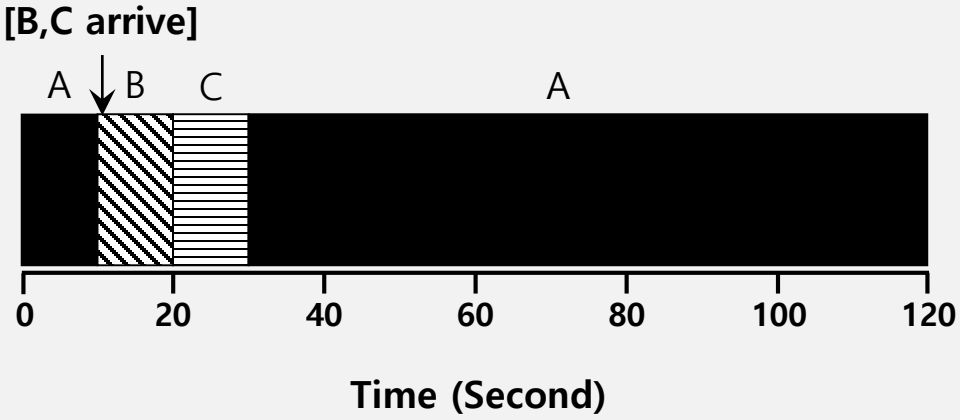
$$\textit{Average turnaround time} = \frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33\ \textit{sec}$$

Shortest Time-to-Completion First (STCF)

- Add preemption to SJF
  - Also knows as Preemptive Shortest Job First (PSJF)
- A new job enters the system:
  - Determine of the remaining jobs and new job
  - Schedule the job which has the lest time left

Shortest Time-to-Completion First (STCF)

- Example:
  - A arrives at t=0 and needs to run for 100 seconds.
  - B and C arrive at t=10 and each need to run for 10 seconds



$$\textbf{Average turnaround time} = \frac{(120-0)+(20-10)+(30-10)}{3} = 50\ sec$$

New scheduling metric: Response time

- The introduction of time-shared machines changed every thing.
- Users would sit at a terminal and demand interactive performance.
- A new metric is born: response time.
- The time from **when the job arrives** to the **first time it is scheduled**.

$$T_{response} = T_{firstrun} - T_{arrival}$$

  - STCF and related disciplines are not particularly good for response time.

How can we build a scheduler that is sensitive to response time?

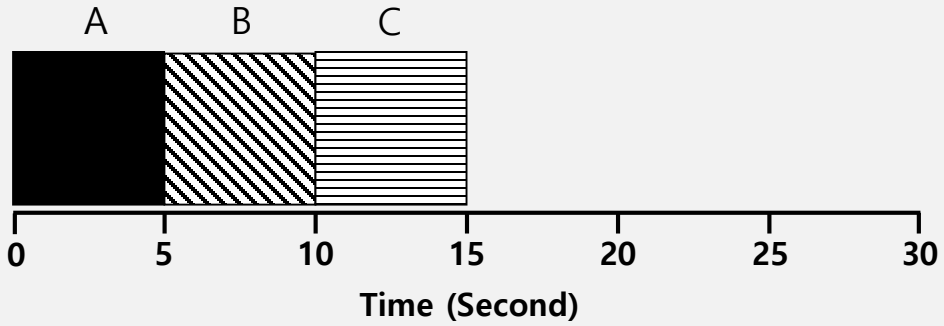
Round Robin (RR) Scheduling

- Time slicing Scheduling
  - Run a job for a time slice and then switch to the next job in the **run queue** until the jobs are finished.
    - Time slice is sometimes called a scheduling quantum.
  - It repeatedly does so until the jobs are finished.
  - The length of a time slice must be *a multiple of* the timer-interrupt period.

RR is fair, but performs poorly on metrics such as turnaround time

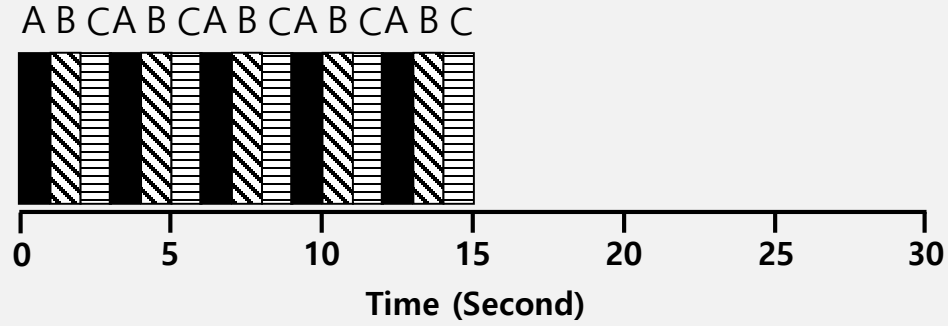
RR Scheduling Example

- A, B and C arrive at the same time.
- They each wish to run for 5 seconds.



SJF (Bad for Response Time)

$$T_{average\ response} = \frac{0 + 5 + 10}{3} = 5sec$$



RR with a time-slice of 1sec (Good for Response Time)

$$T_{average\ response} = \frac{0 + 1 + 2}{3} = 1sec$$

The length of the time slice is critical.

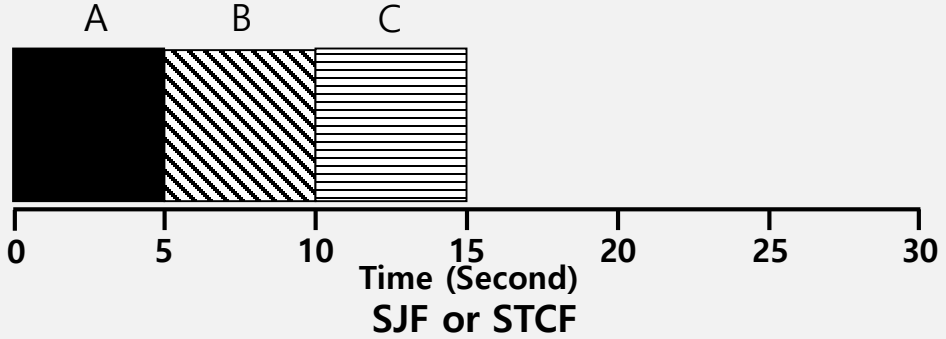
- The shorter time slice
  - Better response time
  - The cost of context switching will dominate overall performance.

- The longer time slice
  - Amortize the cost of switching
  - Without making it so long that the system is no longer responsive.

Deciding on the length of the time slice presents a trade-off to a system designer

The Trade-off:

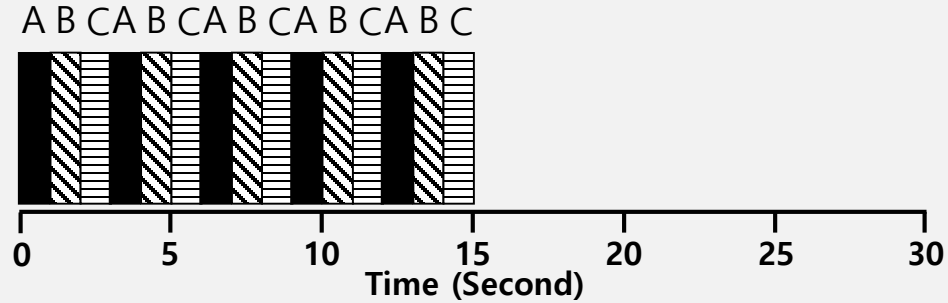
- If you are willing to be unfair, you can run shorter jobs to completion, but at the cost of response time; if you instead value fairness, response time is lowered, but at the cost of turnaround time.



SJF or STCF

$$T_{response} = \frac{0 + 5 + 10}{3} = 5sec$$

$$T_{turnaround} = \frac{5 + 10 + 15}{3} = 10sec$$



RR with a time-slice of 1sec

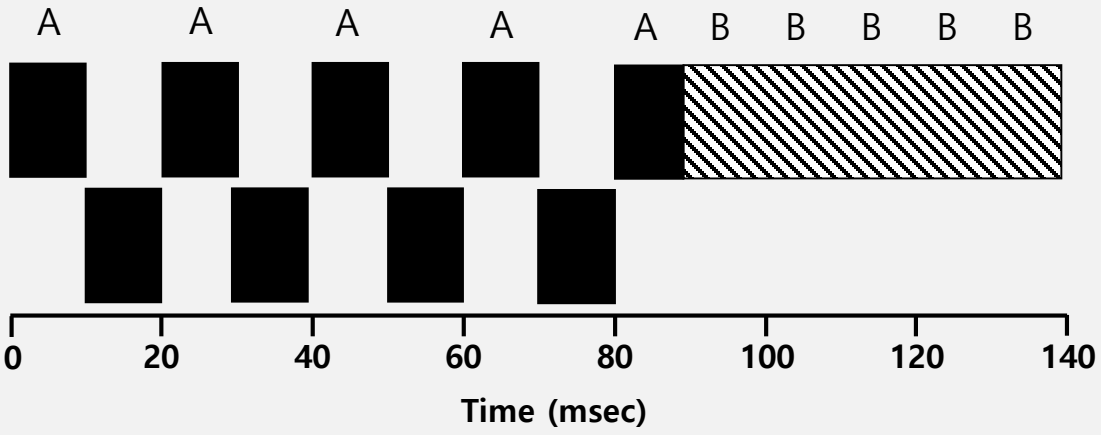
$$T_{response} = \frac{0 + 1 + 2}{3} = 1sec$$

$$T_{turnaround} = \frac{13 + 14 + 15}{3} = 14\ sec$$

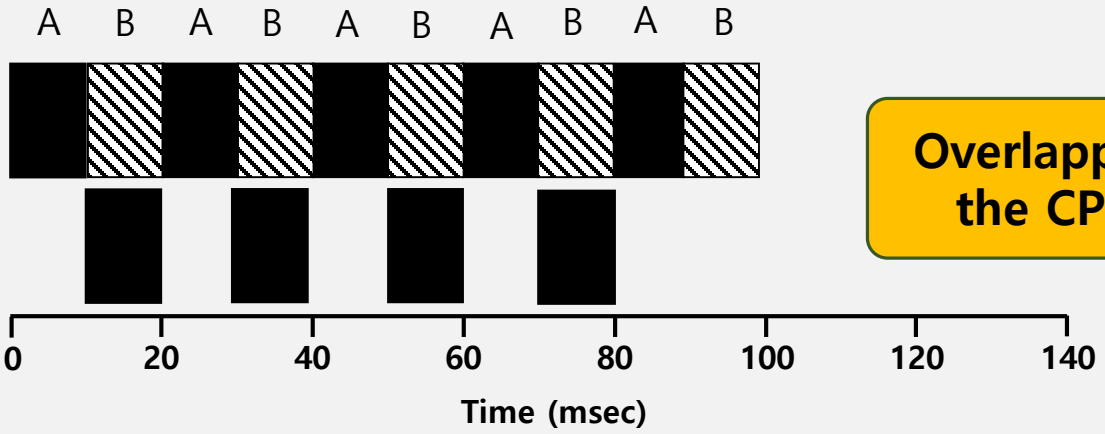
Incorporating I/O

- Let's relax assumption 3: All programs perform I/O
- Example:
  - A and B need 50ms of CPU time each.
  - A runs for 10ms and then issues an I/O request
    - I/Os each take 10ms
  - B simply uses the CPU for 50ms and performs no I/O
  - The scheduler runs A first, then B after

Incorporating I/O (Cont.)



Poor Use of Resources



Overlap Allows Better Use of Resources

Overlapping maximize the CPU utilization

Incorporating I/O (Cont.)

- When a job initiates an I/O request.
  - The job is blocked waiting for I/O completion.
  - The scheduler should schedule another job on the CPU.
- When the I/O completes
  - An interrupt is raised.
  - The OS moves the process from blocked back to the ready state.

8: Scheduling:
The Multi-Level Feedback Queue

Multi-Level Feedback Queue (MLFQ)

- A Scheduler that learns from the past to predict the future.
- Objective:
  - Optimize **turnaround time** → Run shorter jobs first
  - Minimize **response time** without *a priori knowledge of job length*.

MLFQ: Basic Rules

- MLFQ has a number of distinct **queues**.
  - Each queues is assigned a different priority level.

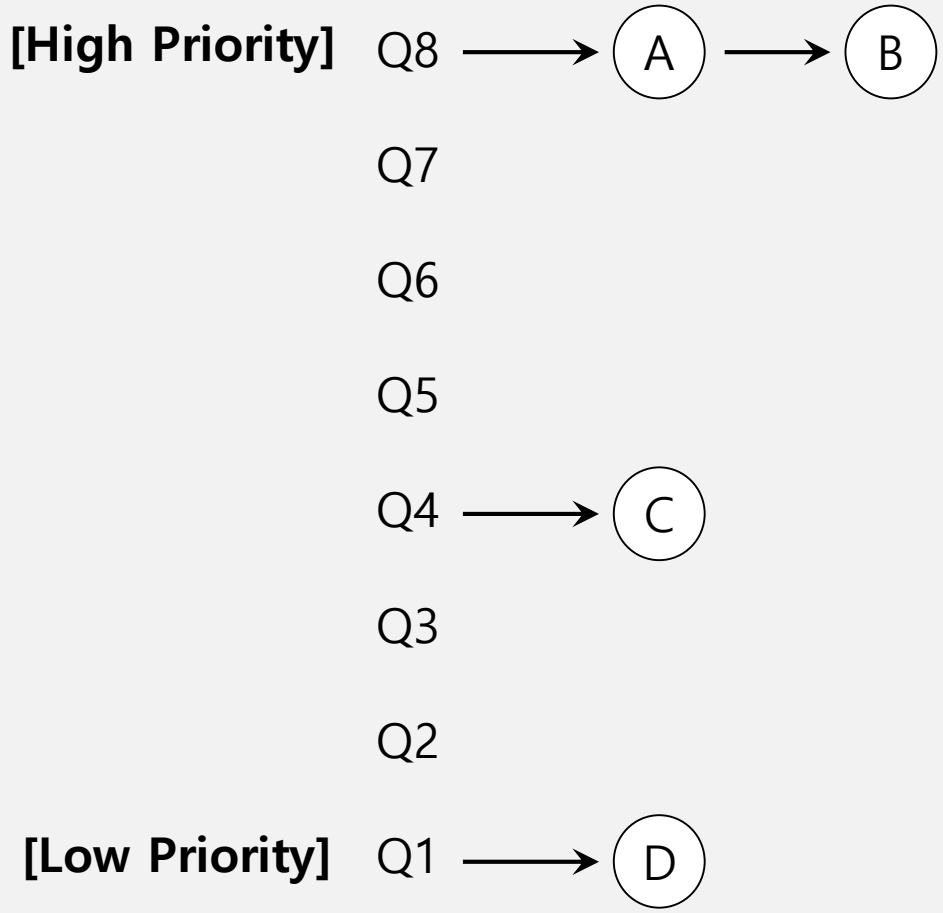
- A job that is ready to run is on a single queue.
  - A job **on a higher queue** is chosen to run.
  - Use round-robin scheduling among jobs in the same queue.

Rule 1: If Priority(A) > Priority(B), A runs (B doesn't).
Rule 2: If Priority(A) = Priority(B), A & B run in RR.

MLFQ: Basic Rules (Cont.)

- MLFQ varies the priority of a job based on its observed behavior.
- Example:
  - A job repeatedly relinquishes the CPU while waiting IOs → Keep its priority high
  - A job uses the CPU intensively for long periods of time → Reduce its priority.

MLFQ Example



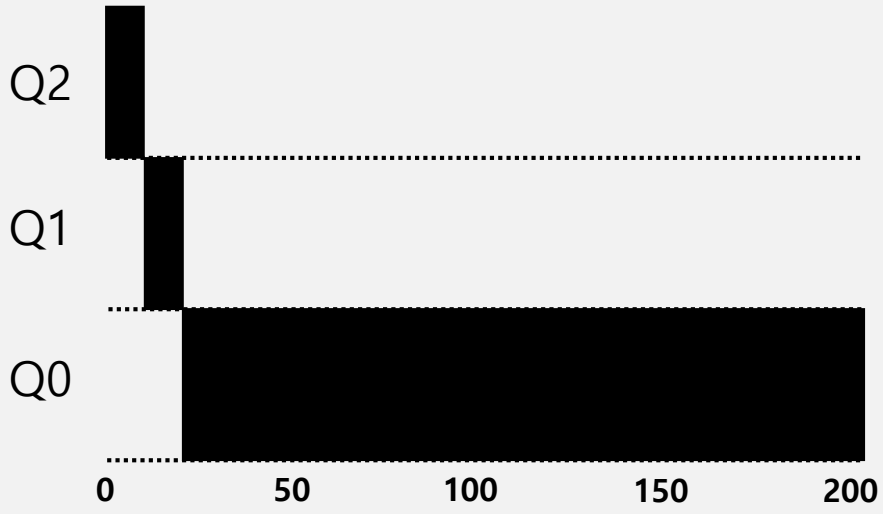
MLFQ: How to Change Priority

- MLFQ priority adjustment algorithm:
  - **Rule 3**: When a job enters the system, it is placed at the highest priority
  - **Rule 4a**: If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down on queue).
  - **Rule 4b**: If a job gives up the CPU before the time slice is up, it stays at the same priority level

In this manner, MLFQ approximates SJF

Example 1: A Single Long-Running Job

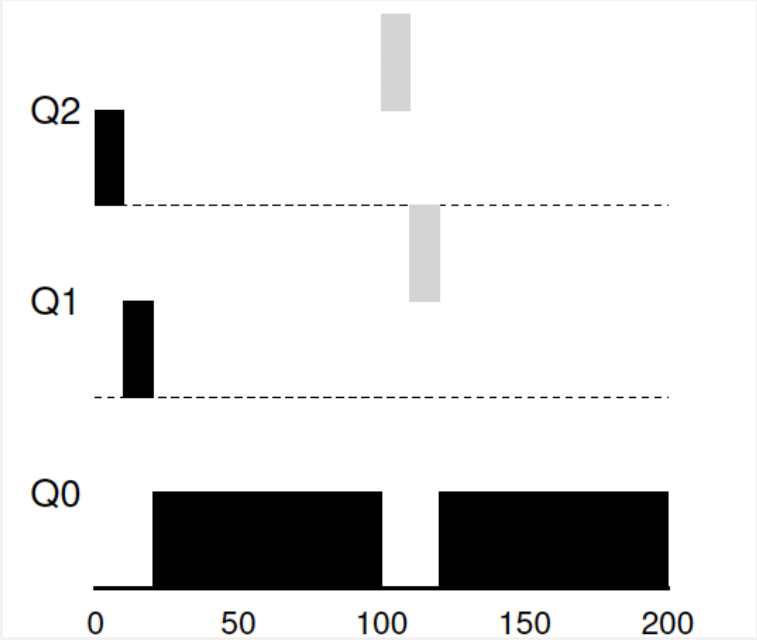
- A three-queue scheduler with time slice 10ms



Long-running Job Over Time (msec)

Example 2: Along Came a Short Job

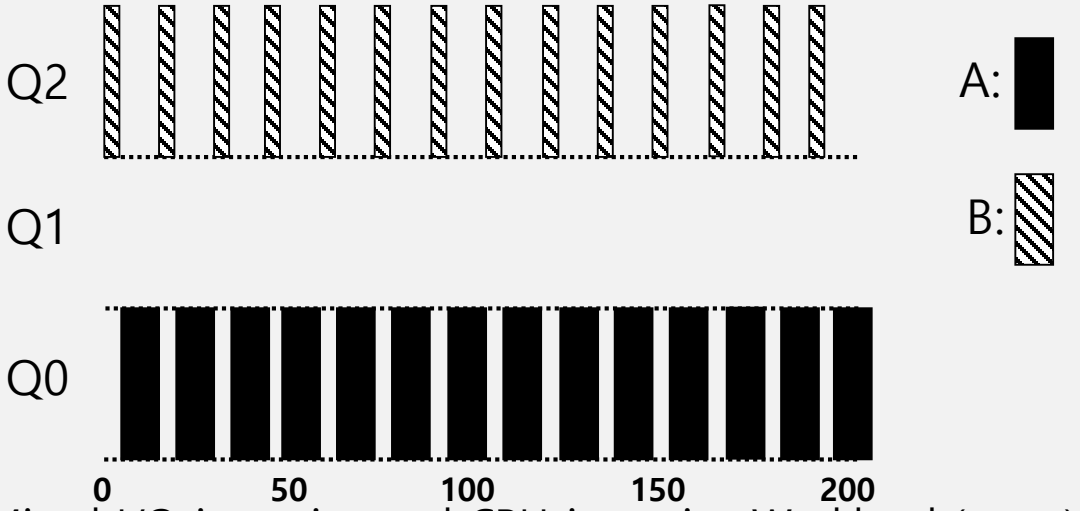
- Assumption:
  - **Job A**: A long-running CPU-intensive job
  - **Job B**: A short-running interactive job (20ms runtime)
  - A has been running for some time, and then B arrives at time T=100.



Along Came An Interactive Job (msec)

Example 3: What About I/O?

- Assumption:
  - **Job A**: A long-running CPU-intensive job
  - **Job B**: An interactive job that need the CPU only for 1ms before performing an I/O



A Mixed I/O-intensive and CPU-intensive Workload (msec)

The MLFQ approach keeps an interactive job at the highest priority

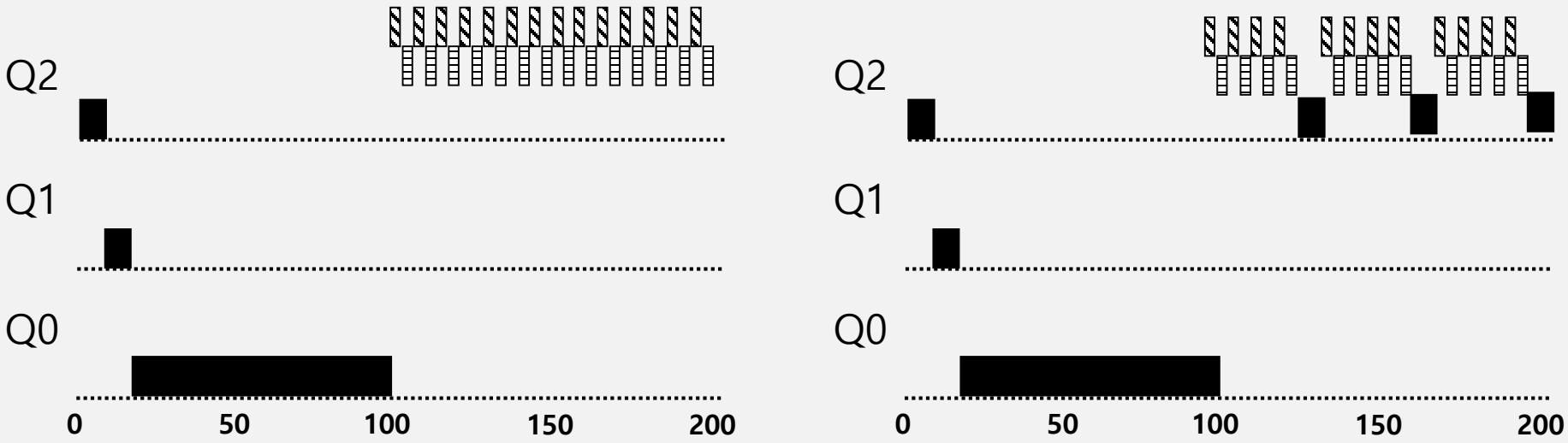
Problems with the Basic MLFQ

- Starvation
  - If there are “too many” interactive jobs in the system.
  - Lon-running jobs will never receive any CPU time.
- Game the scheduler
  - After running 99% of a time slice, issue an I/O operation.
  - The job gain a higher percentage of CPU time.
- A program may change its behavior over time.
  - CPU bound process → I/O bound process

The Priority Boost

Two problems are solved at once:
- ✓ **starvation**
- ✓ **behavior change**

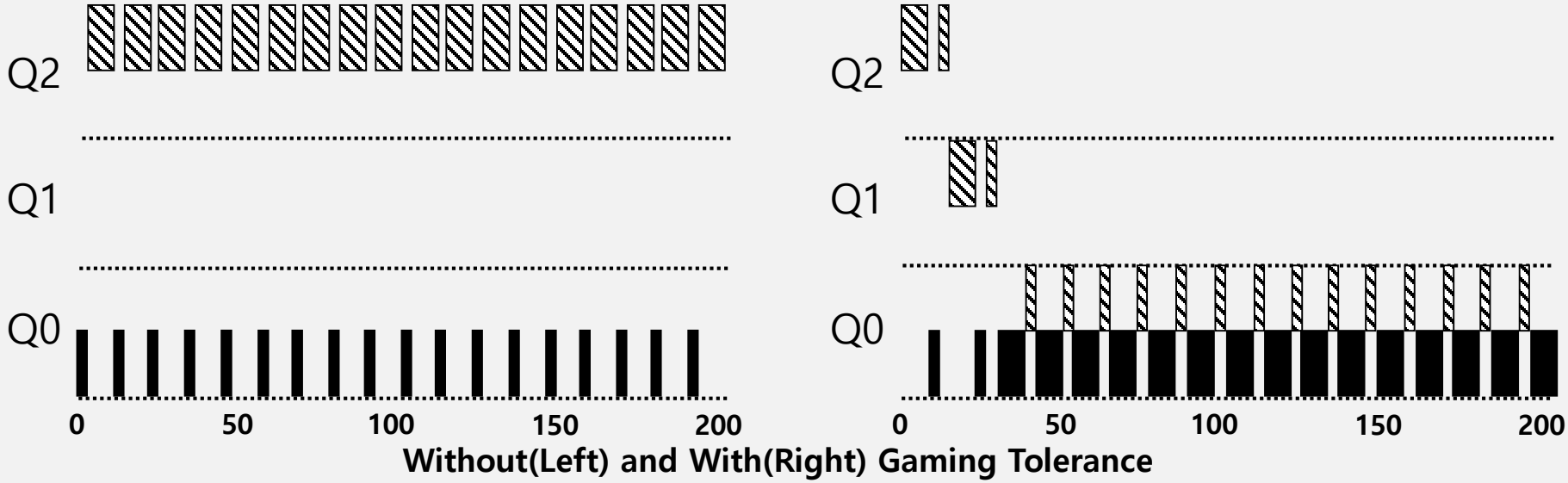
- **Rule 5:** After some time period S, move all the jobs in the system to the topmost queue.
  - Example:
    - A long-running job(A) with two short-running interactive job (B, C)



Without(Left) and With(Right) Priority Boost A: B: C:

Better Accounting

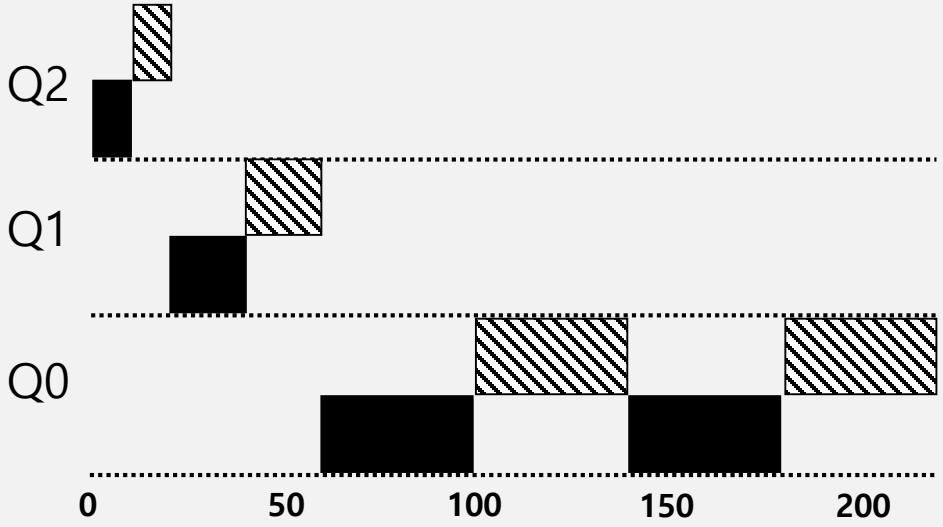
- How to prevent gaming of our scheduler?
- Solution:
  - **Rule 4** (Rewrite Rules 4a and 4b): Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), **its priority is reduced**(i.e., it moves down on queue).



Tuning MLFQ And Other Issues

Lower Priority, Longer Quanta

- The high-priority queues → Short time slices
  - E.g., 10 or fewer milliseconds
- The Low-priority queue → Longer time slices
  - E.g., 100 milliseconds



Example) 10ms for the highest queue, 20ms for the middle, 40ms for the lowest

The Solaris MLFQ implementation

- For the Time-Sharing scheduling class (TS)
  - 60 Queues
  - Slowly increasing time-slice length
    - The highest priority: 20msec
    - The lowest priority: A few hundred milliseconds
  - Priorities boosted around every 1 second or so.

FreeBSD Scheduler(4.3)

- MLFQ without queue.
- Instead, use formula.
- Compute the priority of a process based upon
  - How much CPU a process has used.
  - Boost priority by decay.
  - Take the advice from the user (`nice`).
- For efficiency, use queue.

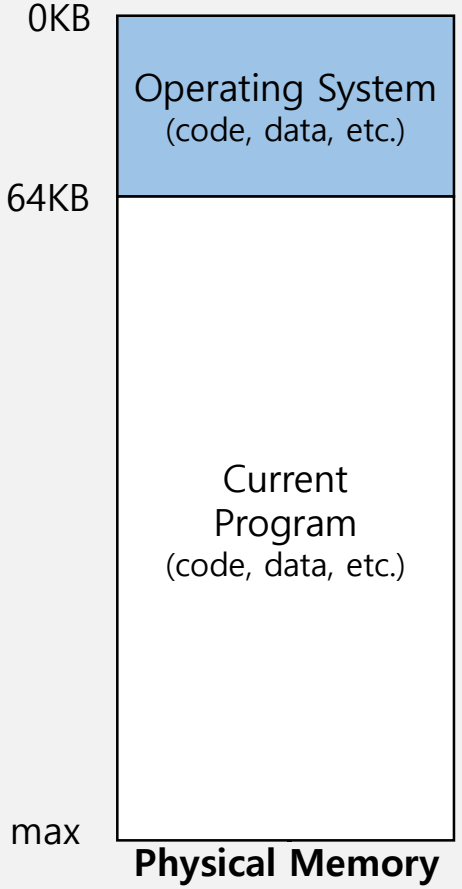
MLFQ: Summary

- The refined set of MLFQ rules:
  - **Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't).
  - **Rule 2:** If Priority(A) = Priority(B), A & B run in RR.
  - **Rule 3:** When a job enters the system, it is placed at the highest priority.
  - **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced(i.e., it moves down on queue).
  - **Rule 5:** After some time period S, move all the jobs in the system to the topmost queue.
- Beauty of MLFQ
  - It does not require prior knowledge on the CPU usage of a process.

13. The Abstraction: Address Space

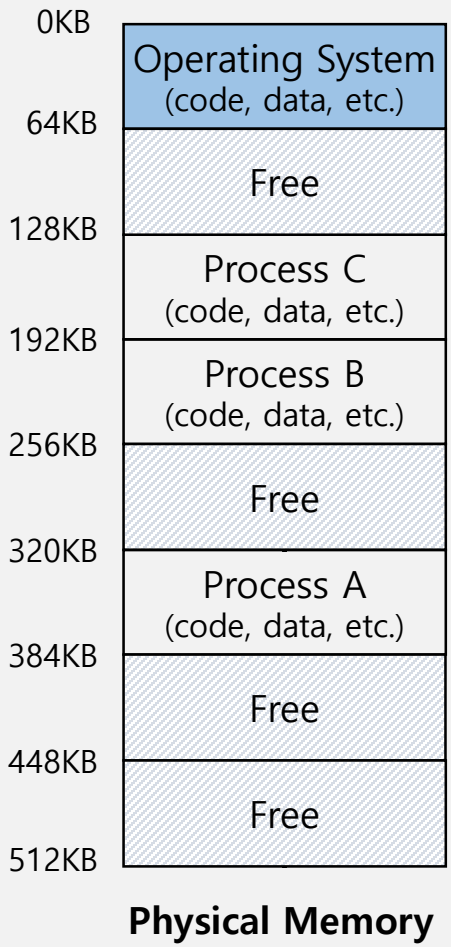
OS in The Early System

- From the perspective of memory, early machines didn't provide much of an abstraction to users.
- The OS was a set of routines, a library, that sat in memory.
- Load only one process in memory.
  - Poor utilization and efficiency



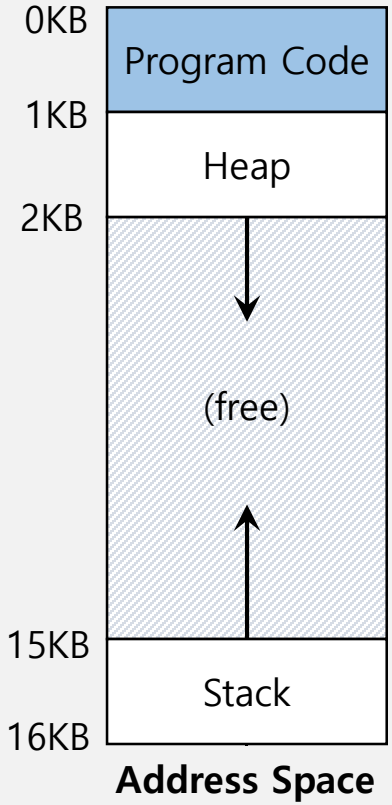
Multiprogramming and Time Sharing

- Because machines were expensive, people began to share machines more effectively.
- **Multiprogramming:** Load multiple processes in memory.
  - Execute one for a short while.
  - Switch processes between them in memory.
  - Increase utilization and efficiency.
- People began demanding more of machines (**time sharing**).
- The notion of **interactivity** become important.



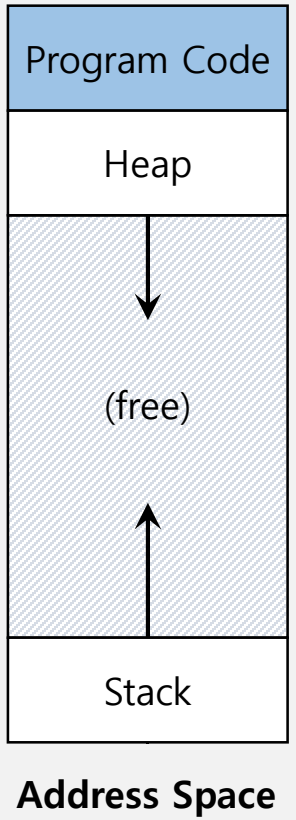
Address Space

- OS creates an **abstraction** of physical memory.
  - It is the running program's view of memory in the system.
  - That is consist of all of memory states of the running program:
    - program code, heap, stack and etc.



Address Space(Cont.)

- Code
  - Where instructions live
- Heap
  - Dynamically allocate memory.
    - `malloc` in C language
    - `new` in object-oriented language
- Stack
  - Store return addresses or values.
  - Contain local variables arguments to routines.



Address Space

Virtual Address

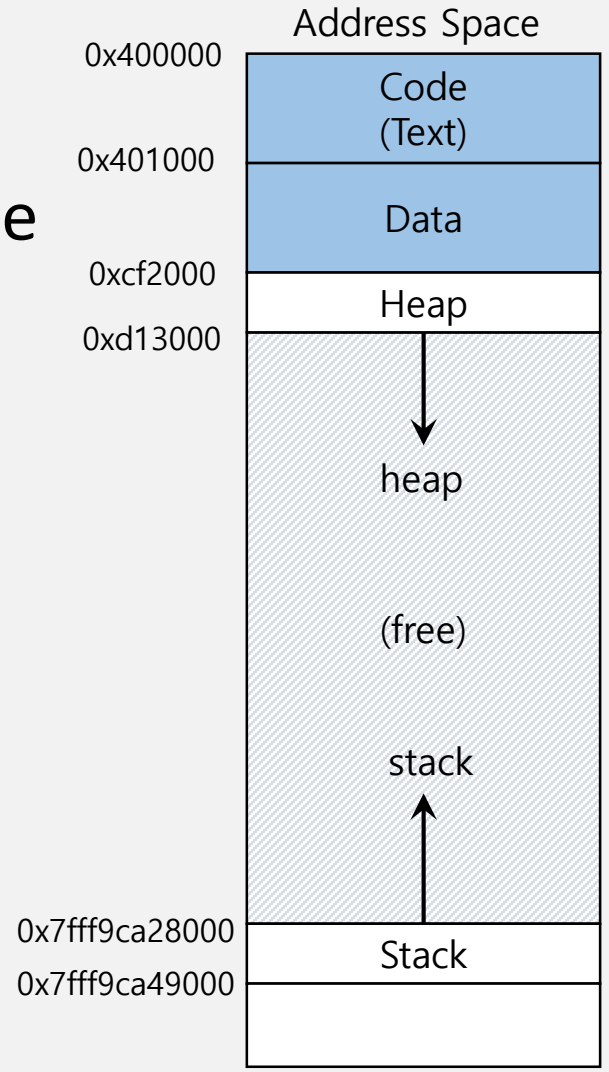
- **Every address** in a running program is virtual.
  - OS translates the virtual address to physical address

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){

    printf("location of code  : %p\n", (void *) main);
    printf("location of heap  : %p\n", (void *) malloc(1));
    int x = 3;
    printf("location of stack : %p\n", (void *) &x);

    return x;
}
```

A simple program that prints out addresses

Virtual Address(Cont.)

- The output in 64-bit Linux machine



Memory Virtualization

- What is **memory virtualization**?
  - OS virtualizes its physical memory.
  - OS provides an illusion memory space per each process.
  - It seems to be seen like each process uses the whole memory .

Benefit of Memory Virtualization

- Ease of use in programming
- Memory efficiency in terms of times and space
- The guarantee of isolation for processes as well as OS
  - Protection from errant accesses of other processes

15. Address Translation

Memory Virtualizing with Efficiency and Control

- Memory virtualizing takes a similar strategy known as **limited direct execution(LDE)** for efficiency and control.
- In memory virtualizing, efficiency and control are attained by hardware support.
  - e.g., registers, TLB(Translation Look-aside Buffer)s, page-table

Address Translation

- Hardware transforms a **virtual address** to a **physical address**.
  - The desired information is actually stored in a physical address.

- The OS must get involved at key points to set up the hardware.
  - The OS must manage memory to judiciously intervene.

Example: Address Translation

```
void func()
        int x=3000;
        ...
        x = x + 3; // this is the line of code we are interested in
```

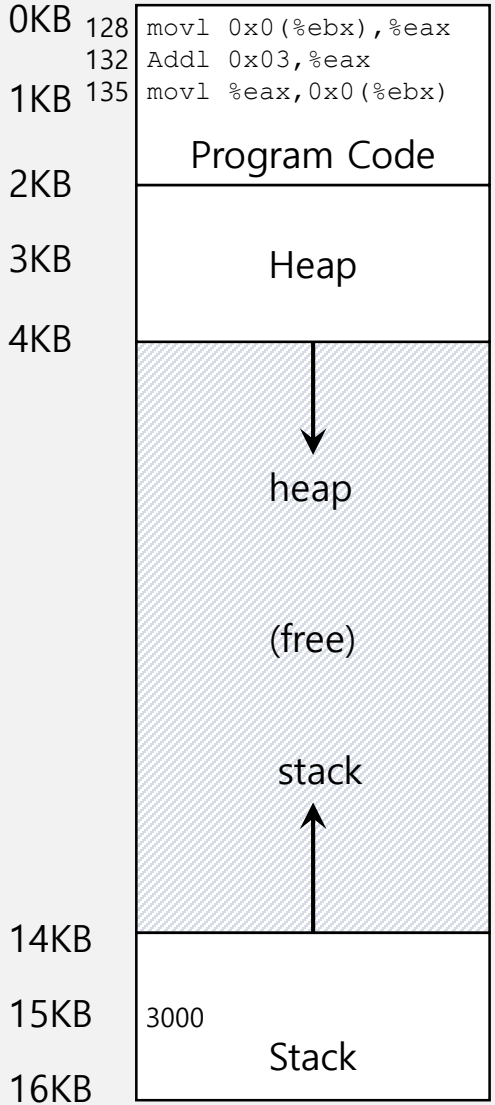
- **Load** a value from memory
- **Increment** it by three
- **Store** the value back into memory

Example: Address Translation(Cont.)

```
128 : movl 0x0(%ebx), %eax          ; load 0+ebx into eax
132 : addl $0x03, %eax              ; add 3 to eax register
135 : movl %eax, 0x0(%ebx)          ; store eax back to mem
```

- Presume that the address of `'x'` has been place in `ebx` register.
- **Load** the value at that address into `eax` register.
- **Add** 3 to `eax` register.
- **Store** the value in `eax` back into memory.

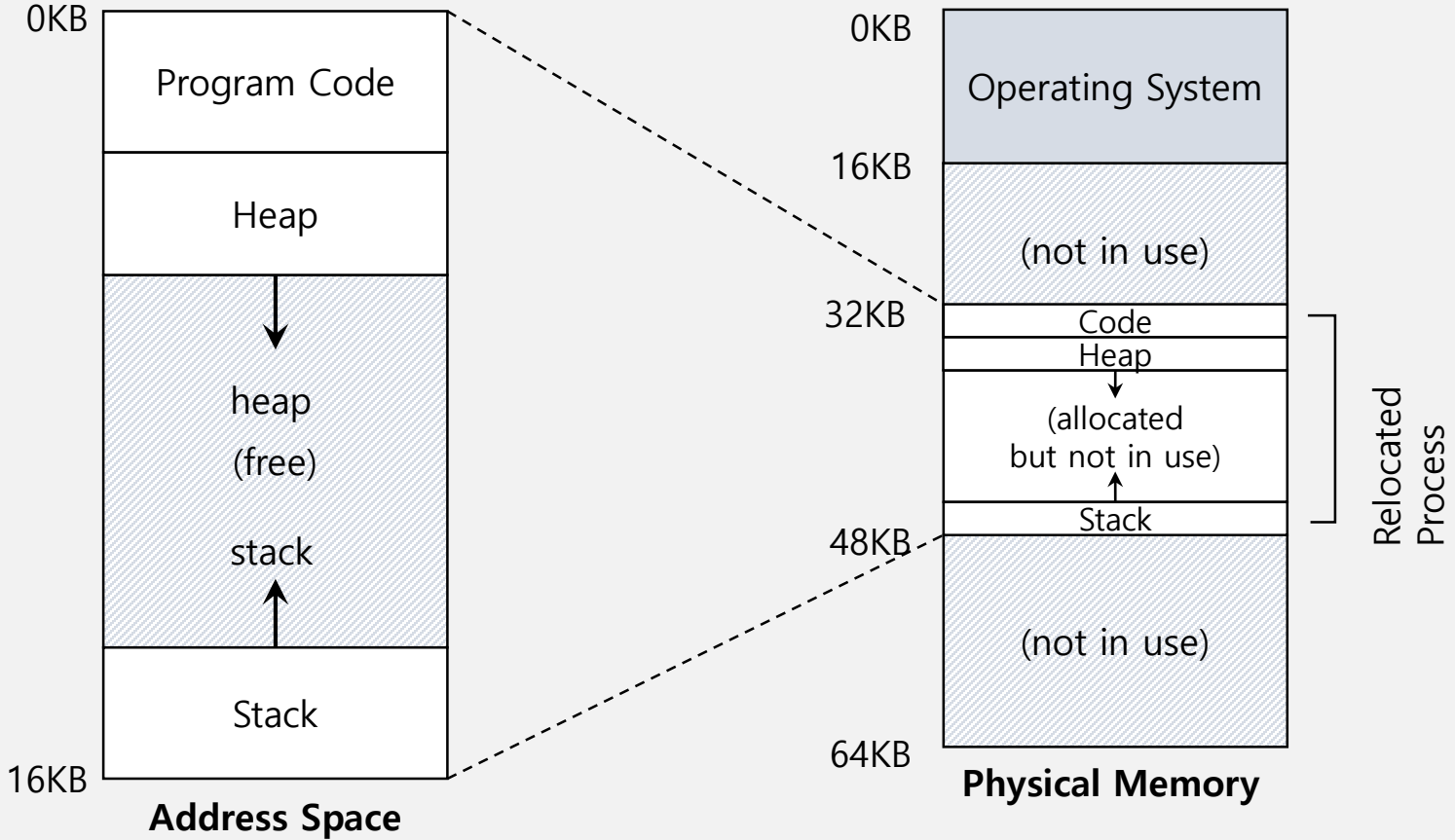
Example: Address Translation(Cont.)



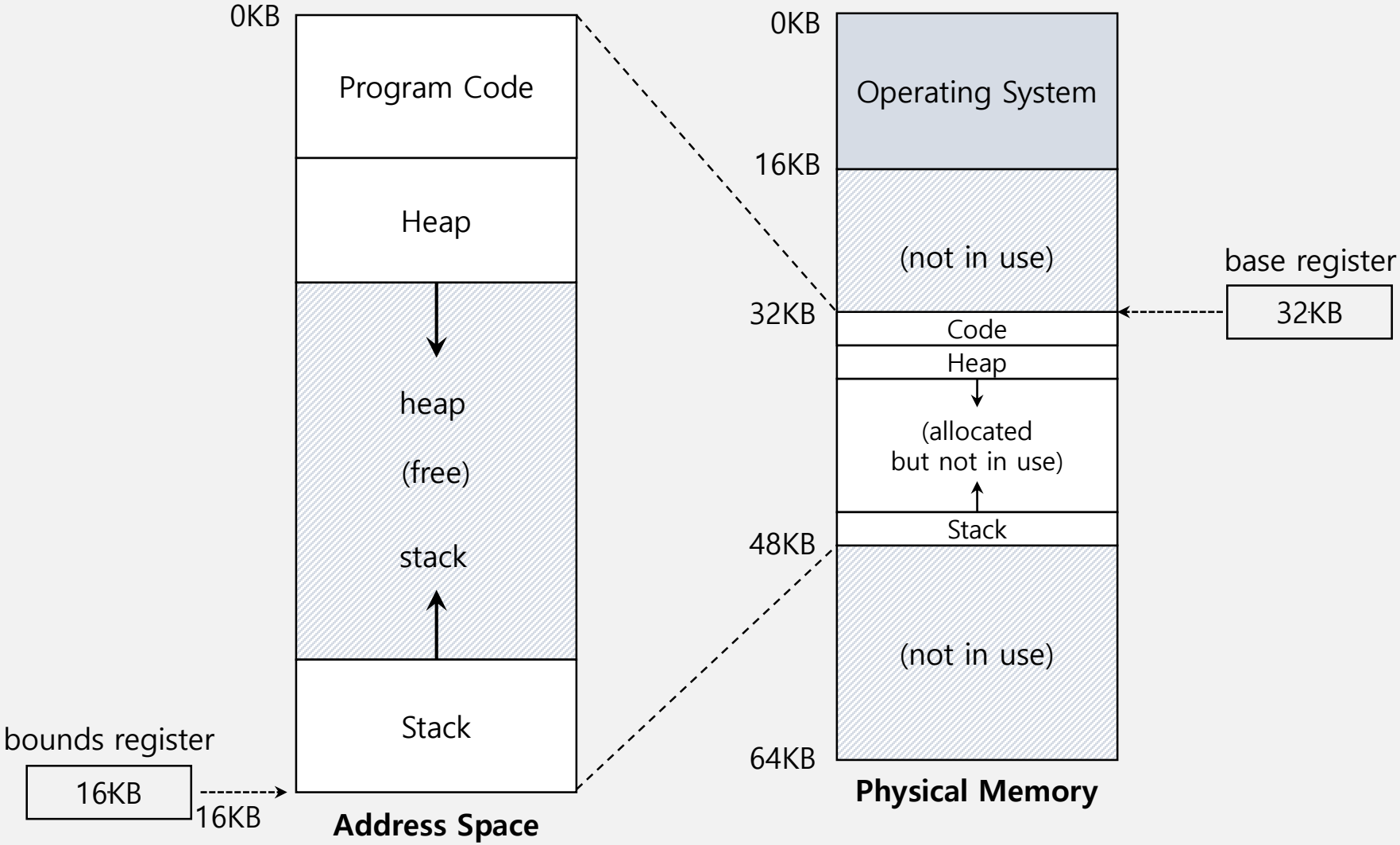
- Fetch instruction at address 128
- Execute this instruction (load from address 15KB)
- Fetch instruction at address 132
- Execute this instruction (no memory reference)
- Fetch the instruction at address 135
- Execute this instruction (store to address 15 KB)

Dynamic Relocation: Base and Bound Register

- The OS wants to place the process **somewhere else** in physical memory, not at address 0.
  - The address space start at address 0.



Base and Bounds Register



Dynamic(Hardware base) Relocation

- When a program starts running, the OS decides **where** in physical memory a process should be **loaded**.
  - Set the **base** register a value.

$$phycal\ address = virtual\ address + base$$

  - Every virtual address must **not be greater than bound** and **negative.**

$$0 \leq\ virtual\ addressvirtual\ address < bounds$$

Relocation and Address Translation

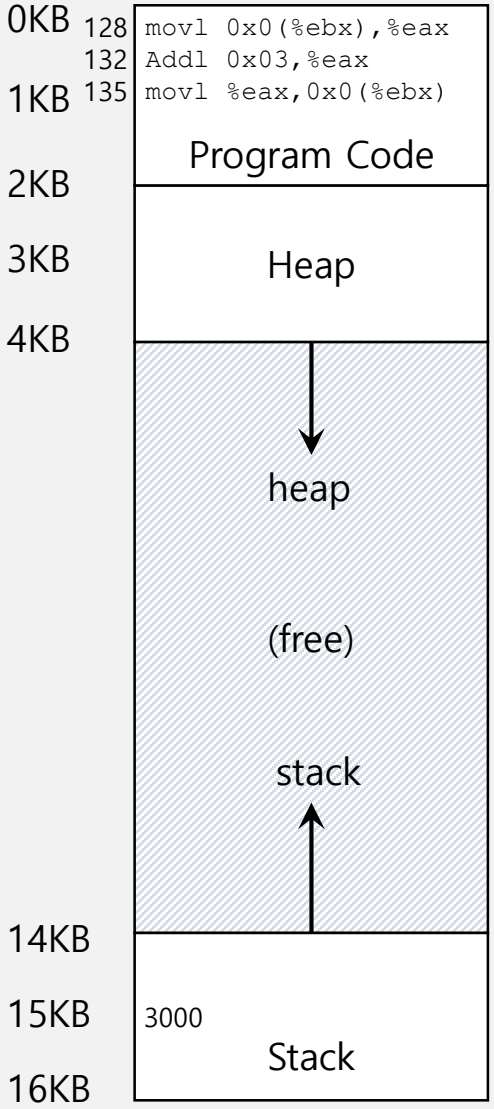
```
128 : movl 0x0(%ebx), %eax
```

- **Fetch** instruction at address 128

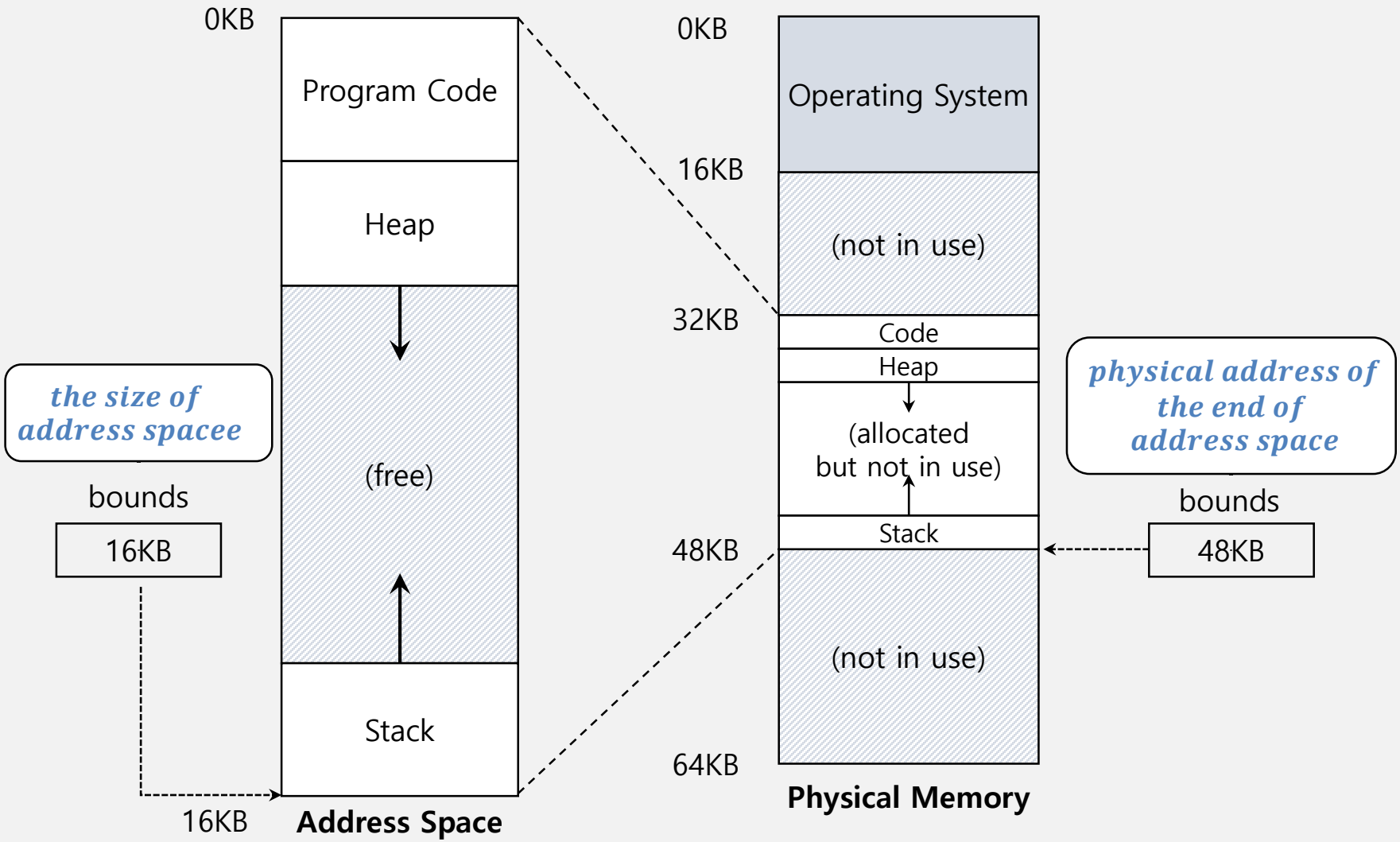
$$32896 = 128 + 32KB(base)$$

- **Execute** this instruction
  - Load the value from address 15KB

$$47KB = 15KB + 32KB(base)$$



Two ways of Bounds Register



Hardware Requirements

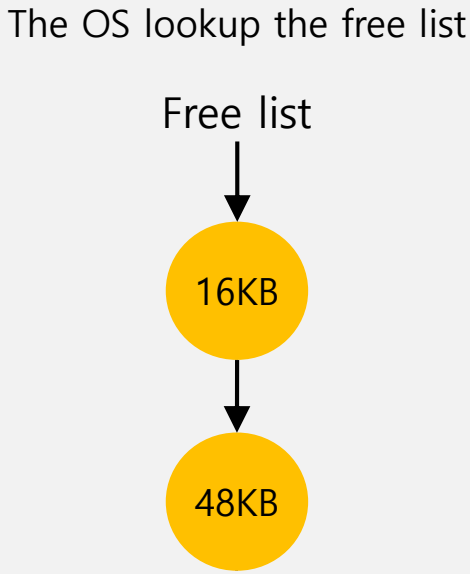
- Privileged mode: prevent user-mode processes from executing privileged operations
- Base/bounds registers: Need pair of registers per CPU to support address translation and bounds checks
- Ability to translate virtual addresses and check if within bounds limits; Circuitry to do translations.
- Privileged instruction(s) to update base/bounds: OS must be able to set these values before letting a user program run
- Privileged instruction(s) to register: OS must be able to tell hardware what exception handlers code to run if exception occurs
- Ability to raise exceptions when processes try to access privileged instructions or out-of-bounds memoryl

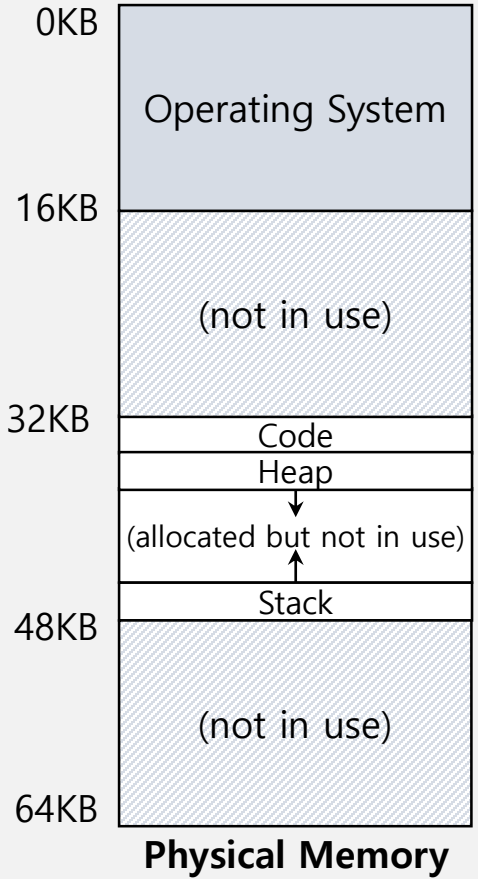
OS Issues for Memory Virtualizing

- The OS must **take action** to implement **base-and-bounds** approach.
- Three critical junctures:
  - When a process **starts running:**
    - Finding space for address space in physical memory
  - When a process is **terminated:**
    - Reclaiming the memory for use
  - When context **switch occurs:**
    - Saving and storing the base-and-bounds pair

OS Issues: When a Process Starts Running

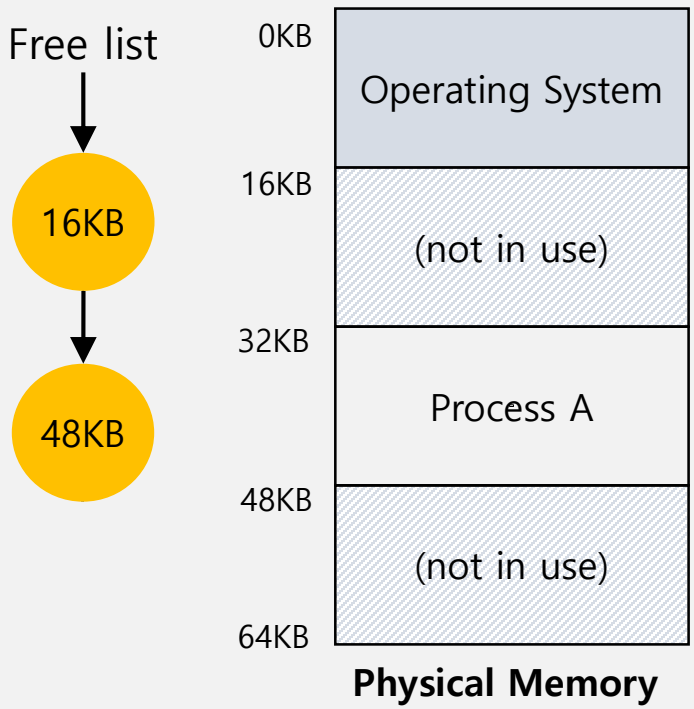
- The OS must **find a room** for a new address space.
  - free list : A list of the range of the physical memory which are not in use.

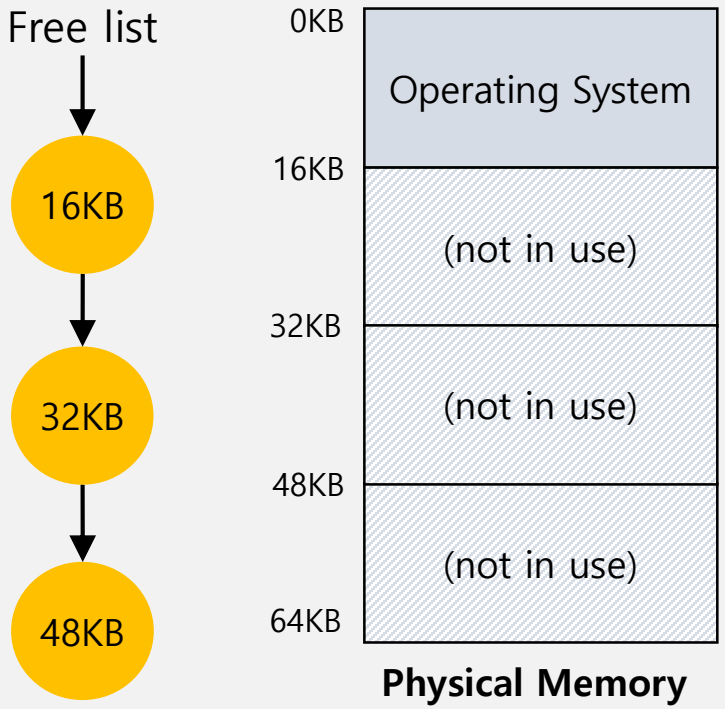




OS Issues: When a Process Is Terminated

- The OS must **put the memory back** on the free list.





OS Issues: When Context Switch Occurs

- The OS must **save and restore** the base-and-bounds pair.
  - In **process structure** or **process control block(**PCB)